Тема 6: PHP. Работа с БД

Подготовительный этап: Установка сервера Apache и PHP

В качестве программного обеспечения предлагается использовать дистрибутив объёмом 5,43 Мб (Denwer3_Base_2008-01-13_a2.2.4_p5.2.4_m5.0.45_pma2.6.1),который можно бесплатно взять на сайте разработчика http://denwer.ru. В развёрнутом виде на HDD (рекомендуемый каталог установки C:\WebServers) потребуется < 23 Мб.

Состав базового пакета Денвера:

1. Apache 2 с поддержкой SSL и mod_rewrite.
2. PHP5: выполняемые файлы, модуль для веб-сервера Apache, дистрибутивный и адаптированный конфигурационный файл, библиотека GD, модули поддержки MySQL и sqLite.
3. MySQL5 с поддержкой InnoDB, транзакций и русских кодировок (windows-1251).
4. phpMyAdmin - панель управления базой данных MySQL, а также скрипт, упрощающий добавление нового пользователя MySQL.
5. Отладочный эмулятор sendmail (/usr/sbin/sendmail), не отправляющий письма, а записывающий их в директорию /tmp/!sendmail.
6. Система автоматического поиска виртуальных хостов и обновления системного файла hosts, а также конфигурации Apache. Благодаря ей добавление нового виртуального хоста (или домена третьего уровня) заключается в простом создании каталога в /home (см. по аналогии с уже существующими хостами) и перезапуске комплекса. Все изменения вносятся в конфигурационные и системные файлы автоматически, но возможно управление этим процессом при помощи механизма шаблонов хостов (см. /usr/local/apache/conf/httpd.conf за детальными разъяснениями).

Структура каталогов

HTML-документы должны находиться в директориях /home/<имя_хоста>/www. По умолчанию сконфигурированы 3 виртуальных хоста:

- - http://localhost (содержит скрипты тестирования и различные утилиты);
- - http://test1.ru;
- - http://custom-host:8648 (хост, имеющий свой собственный IP-адрес и порт).

Поддерживаются также виртуальные хосты с доменными именами третьего и выше уровней. Примеры того, как Apache ищет директории документов хостов в этом случае:

Доменное имя	Директория документов
ab.cd.test1.ru	/home/test1.ru/ab.cd
test.localhost	/home/localhost/test
ab.cd.localhost	/home/localhost/ab.cd

Настройка возможности обработки PHP-скриптов в html-файлах

- откройте файл «C:\WebServers\usr\local\apache\conf \httpd.conf»;
- найдите в нём строку, начинающуюся на «AddType application/x-httpd-php»;
- допишите в её конец « .html» (с пробелом впереди);
- перед этой строкой вставьте «RemoveHandler .html»;
- после этого удостоверьтесь, что в директиве «AddHandler server-parsed» (найдите эту строчку) нет расширения «.html» (если есть — удалите его);
- перезапустите Денвер.

Учтите, что после этой операции файлы с расширением «.html» перестанут обрабатываться SSI.

Общая последовательность действий при взаимодействии с сервером MySQL

1. Установить соединение с сервером MySQL. Если попытка завершается неудачей, вывести соответствующее сообщение и завершить процесс.
2. Выбрать базу данных сервера MySQL. Если попытка выбора завершается неудачей, вывести соответствующее сообщение и завершить процесс. Допускается одновременное открытие нескольких баз данных для обработки запросов.
3. Обработать запросы к выбранной базе (или базам).
4. После завершения обработки запросов закрыть соединение с сервером баз данных.

Важное замечание:

- файлы следует создавать на виртуальном диске (по умолчанию – Z:) в каталоге Z:\home\localhost\www;
- предварительно запустить сервер программой C:\WebServers\denwer\Run.exe либо с помощью соответствующего ярлыка на Рабочем столе;
- проверить корректность установки и работы сервера можно, набрав в окне адреса Internet Explorer строку http://localhost/ и нажав клавишу Enter. Нормальная реакция представлена на рис. 6.1;
- запускать файлы следует, набирая в окне адреса Internet Explorer строку http://localhost/имя_файла.

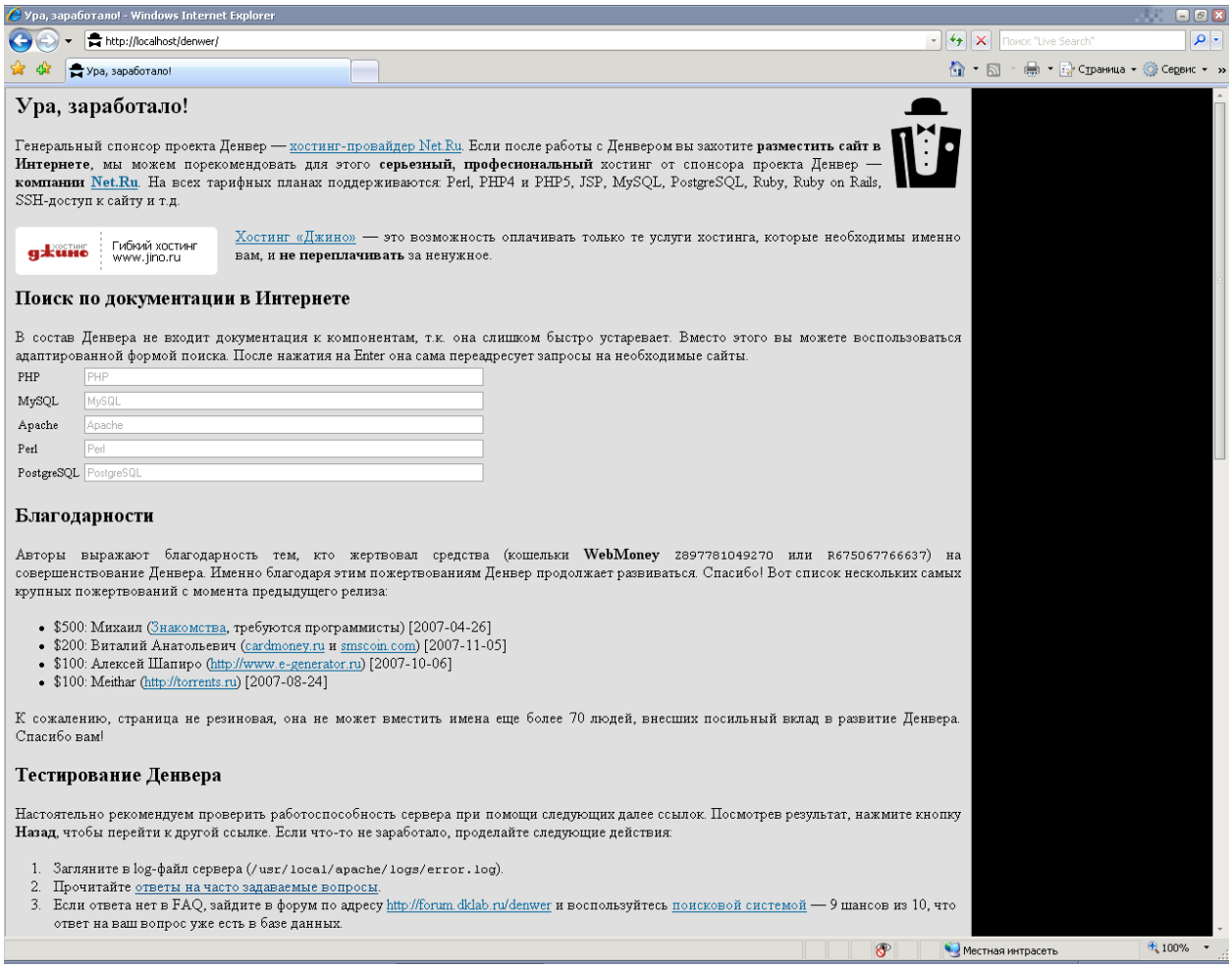

Рис. 6.1

Функции работы с БД

mysql_connect

Устанавливает связь с сервером MySQL После успешного подключения к MySQL можно переходить к выбору баз данных, обслуживаемых этим сервером. Синтаксис:
int mysql_connect ([string хост [:порт] [:/путь//к/сокету] [, string имя пользователя] [, string пароль])

mysql_close

Закрывает установленное ранее соединение с базой данных.

В параметре хост передается имя хостового компьютера, указанное в таблицах привилегий сервера MySQL. Конечно, оно же используется для перенаправления запросов на web-сервер, на котором работает MySQL, поскольку к серверу MySQL можно подключаться в удаленном режиме. Наряду с именем хоста могут указываться необязательные параметры — номер порта, а также путь к сокету (для локального хоста). Параметры имя_пользователя и пароль должны соответствовать имени пользователя и паролю, заданным в таблицах привилегий MySQL. Обратите внимание: все параметры являются необязательными, поскольку таблицы привилегий можно настроить таким образом, чтобы они допускали соединение без проверки. Если параметр хост не задан, mysql_connect( ) пытается установить связь с локальным хостом.

Пример:

Исхолный код	Код, возвращаемый пользователю
<html> <title> <? echo"MySQL connect "; ?></title> <body> <? @mysql_connect("localhost", "root", "a914") or die("Could not connect to MySQL server!"); print "You're connected to MySQL !"; mysql_close(); ?> </body> </html>	<html> <title>MySQL connect </title> <body> You're connected to MySQL ! </body> </html>

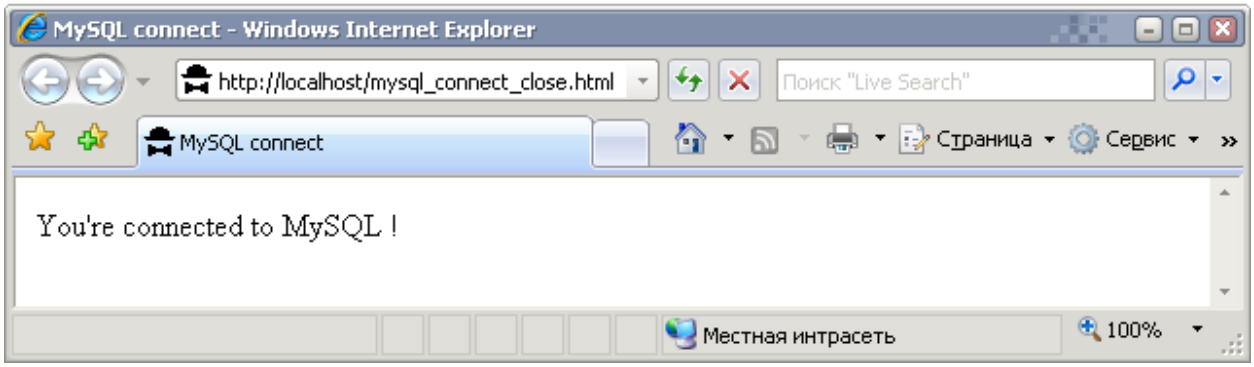

Рис. 6.2

В данном примере localhost — имя компьютера, root— имя пользователя, a914 — пароль. Знак @ перед вызовом функции mysql_connect( ) подавляет все сообщения об ошибках, выдаваемые при неудачной попытке подключения, — они заменяются сообщением, указанным при вызове die( ).

mysql_pconnect

Устанавливает устойчивое сетевое соединение с базой данных MySQL. Синтаксис **:**

int mysql_pconnect([string $hostname[:port][:/path/to/socket][, [,string $username [,string $password]]])

Устанавливает устойчивое сетевое соединение с базой данных MySQL, расположенной на хосте *$hostname*, и возвращает идентификатор открытого соединения. Вся дальнейшая работа ведется именно с этим идентификатором. При регистрации указывается имя пользователя *$username* и пароль *$password*. Строка *$hostname* также может включать в себя номер порта в виде "hostname:port"или путь к сокету для локальной машины в системах Unix - ":/path/to/socket" (если сервер MySQL настроен не на станадартный, а на какой-то другой порт). При ошибке выдается предупреждение. Выдачу сообщения об ошибке можно блокировать, указав перед именем функции оператор "@". При следующем запуске функции с теми же самыми аргументами второе соединение не будет открыто, а функция возвратит идентификатор уже существующего. **mysql_pconnect()** действует аналогично **mysql_connect**, но с двумя отличиями:

- перед подключением функция пытается проверить, имеется ли уже открытое подключение. Если есть, то возвращается идентификатор вместо создания нового подключения;
- при завершении сценария подключение не закрывается, а остается действующим для дальнейшего использования, т.е. функция **mysql_close()** не может закрыть подключения, созданное с помощью **mysql_pconnect()**.

mysql_change_user

Изменяет параметры подключения. Синтаксис**:**

int mysql_change_user(string user, string password [, string database [, int link_identifier]])

Если не указывается БД или подключение, то используется последняя активная БД. Если авторизация не произошла, то параметры подключения не изменяются. Работает с MySQL 3.23.3 и выше.

mysql_list_dbs

Возвращает список БД на сервере. Синтаксис**:**

int mysql_list_dbs([int link_identifier])

Возврашает набор записей, содержащий список БД на сервере.

```
<html>
<title><? echo"MySQL connect "; ?></title>
<body>
<? $bd=mysql_connect("localhost", "root", "a914");
$bd_list=mysql_list_dbs($bd);
while($row=mysql_fetch_object($bd_list)) {
  echo $row->Database."\n";}
mysql_close();
?>
</body>
</html>
}
```

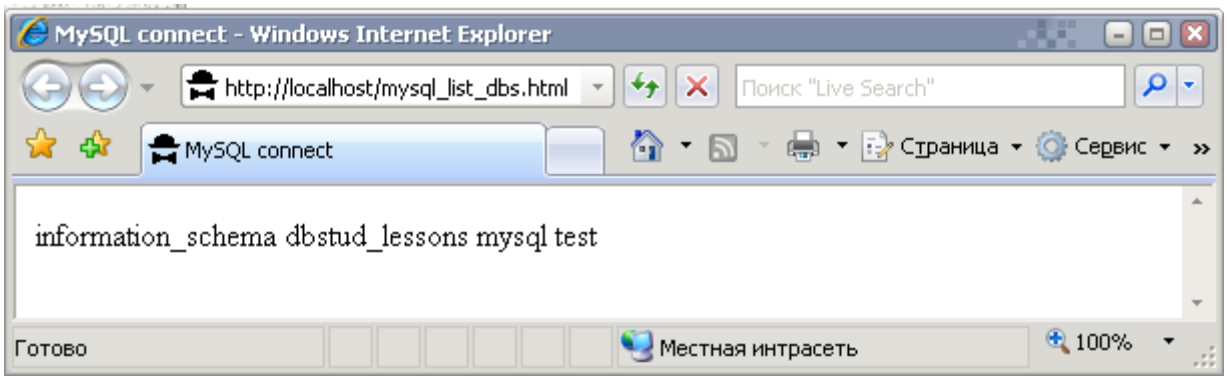

Рис. 6.3

mysql_select_db

Выбор одной базы данных MySQL. Синтаксис**:**

int mysql_select_db (string database_name [, int link_identifier])

Возвращает true при удачном закрытии или false при ошибке. Если планируется открывать только одно соединение с базой данных за все время работы сценария, то можно не сохранять возвращенное значение, а также не указывать идентификатор при вызове всех остальных функций. До того как послать первый запрос серверу MySQL, необходимо указать, с какой базой данных планируется работать. Для этого и предназначена данная функция. Она уведомляет, что в дальнейших операциях с соединением *link_identifier* (или с последним открытым соединением, если указанный параметр не задан) будет использоваться база данных *database_name*. Если на момент вызова данной функции подключений к базе данных нет, то косвенно вызывается функция **mysql_connect()** с параметрами по умолчанию.

mysql_list_tables

Возвращает список таблиц в БД. Синтаксис**:**

int mysql_list_tables(string database [,int link_identifier])

Функция возвращает идентификатор результата (одна колонка), в котором содержаться имена всех таблиц, присутствующих в базе данных. Для извлечения этих имен можно использовать функцию **mysql_result()** с номером колонки, равным 0, или функцию **mysql_tablename()**.

Следующий пример выведет все имена баз данных и таблиц, корорые в них содержаться:

```
<html>
<title><? echo"MySQL connect "; ?></title>
<body>
<? $db=mysql_connect("localhost", "root", "a914");

$db_list=mysql_list_dbs($db);
while($r_db=mysql_fetch_object($db_list)) {
  echo $r_db->Database."<br>";
    // распечатать список таблиц
  $t_list=mysql_list_tables($r_db->Database);
  for($i=0;$i<mysql_num_rows($t_list);$i++)
        { echo " - ".mysql_tablename($t_list,$i)."<br>";  }
}
mysql_close();
?>
</body>
</html>
```

mysql_tablename

Возвращает имя таблицы в БД. Синтаксис**:**

int mysql_tablename(int result, int i)

Функция возвращает имя таблицы с номером *i* из набора записей, полученных при помощи функции **mysql_list_tables()**.

```
<html>
<title><? echo"MySQL connect "; ?></title>
<body>
<? $db=mysql_connect("localhost", "root", "a914");
$result=mysql_list_tables("dbStud_lessons");
$i=0;
while($i<mysql_num_rows($result)) {
  $t_name[$i]=mysql_tablename($result, $i);
  echo $t_name[$i]."<BR>";
  $i++;}
mysql_close();
?>
</body>
</html>
```

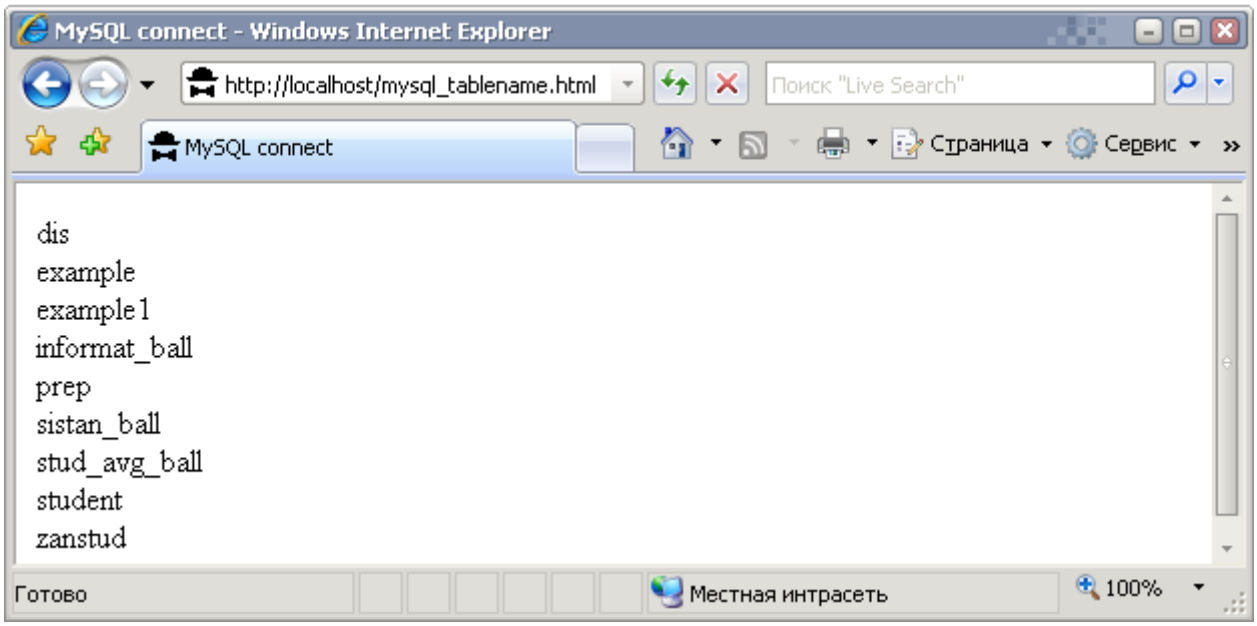

Рис. 6.4

mysql_query

Посылает запрос базе данных MySQL. Синтаксис**:**

int mysql_query(string query [,int link_identifier])

Эта функция посылает запрос *query* базе данных, связанной с идентификатором *link_identifier* Если идентификатор не указан, то принимается во внимание последнее открытое соединение. Если до этого соединение небыло установлено, то выполняется операция **mysql_connect()** с параметрами по умолчанию. SQL-выражение, указанное в параметре *query*, не должно оканчиваться ";".Если выражение содержит ошибки, или его выполнение приводит к ошибкам, то функция **mysql_query()** возвращает false.

В результате успешно выполненного запроса возвращается набор записей, который можно обработать следующими функциями:

- mysql_result() - получить элемент набора записей;

- mysql_fetch_array() - внести запись в массив;
- mysql_fetch_row() - занести запись в нумерованный массив;
- mysql_fetch_assoc() - занести запись в ассоциативный массив;
- mysql_fetch_object() - занести запись в объект.

Чтобы узнать, сколько записей было найдено командой SELECT, воспользуйтесь функцией **mysql_num_rows()**. Для того, чтобы узнать, сколько записей было изменено в результате выполнения запросов DELETE, INSERT, REPLACE или UPDATE, воспользуйтесь функцией **mysql_affected_rows()**. После обработки результатов запроса он может быть удален функцией **mysql_free_result()**. Но в этом нет необходимости, т.к. результаты сами уничтожаются после завершения работы сценария.

mysql_db_query

Посылает запрос к указанной базе данных MySQL. Синтаксис**:**

int mysql_db_query(string database, string query [,int link_identifier])

Эта функция эквивалентна следующей последовательности функций:

mysql_select_db(string database [, int link_identifier]);
mysql_query(string query [, int link_identifier]);

mysql_num_rows

Возвращает количество строк в результате запроса. Синтаксис**:**

int mysql_num_rows(int result)

Эта функция возвращает число записей, найденных в результате выполнения SQL-команды SELECT (поиск по базе данных).

```
<html>
<title><? echo"MySQL connect database"; ?></title>
<body>
<?
mysql_connect("localhost", "root", "a914")
or die("Could not connect to MySQL server!");
mysql_select_db("dbStud_lessons")
  or die("Could not select  database!");
  print "You're connected to a MySQL database   dbStud_lessons !";

 $query = "SELECT * FROM student"; $result = mysql_query($query);
print"<br>";
print " Total rows selected: ".mysql_num_rows($result);

mysql_close();
?>
</body>
</html>
```

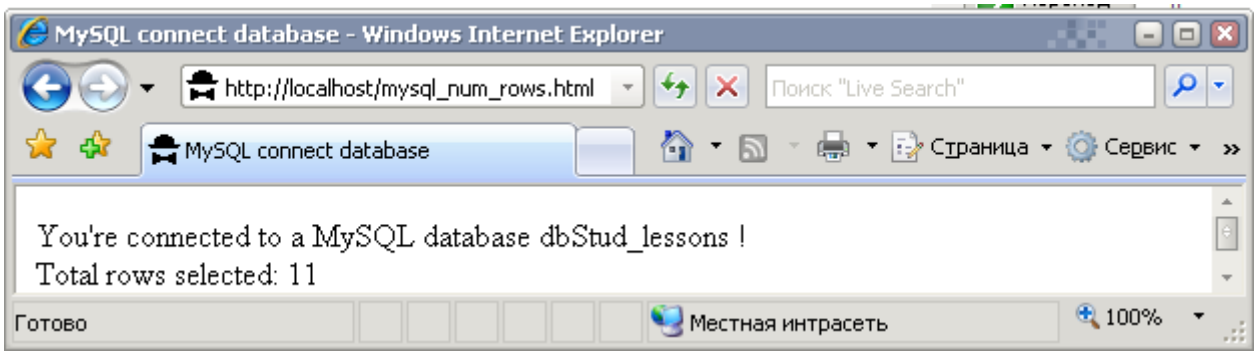

Рис. 6.5

mysql_result( )

Используется в сочетании с mysql_query( ) (при выполнении запроса с командой SELECT) для получения набора данных. Синтаксис функции:

int mysql_result (int идентификатор_результата, int запись [. mixed поле"]')

В параметре идентификатор_результата передается значение, возвращенное функцией mysql_query( ). Параметр запись ссылается на определенную запись набора данных, определяемого параметром идентификатор_результата. Наконец, в необязательном параметре поле могут передаваться:

- смещение поля в таблице;
- имя поля;
- имя поля в формате *имя_поля_имя_тдблицы.*

Пример: выдача содержимого таблицы Студенты БД dbStud_lessons.

```
<html>
<title><? echo"MySQL connect database"; ?></title>
<body>
<?
@mysql_connect("localhost", "root", "a914")
or die("Could not connect to MySQL server!");
$conn = mysql_select_db("dbStud_lessons")
  or die("Could not select  database!");
  print "You're connected to a MySQL database   dbStud_lessons !";

 $query = "SELECT * FROM student"; $result = mysql_query($query);
$x=0;
print"<br><h3>Table Students</h3>";
print "<table>";
print"<tr> <th>KodStud </th><th>Fam </th><th>Imya
</th><th>Otch</th><th>Gr_nomber </th></tr>";
while ($x < mysql_num_rows($result)):
     $kodStud=mysql_result($result, $x, 'kodStud');
     $fam=mysql_result($result, $x, 'fam');
     $imya=mysql_result($result, $x, 'imya');
    $otch=mysql_result($result, $x, 'otch');
    $gr_nomber=mysql_result($result, $x, 'gr_nomber');
    print"<tr>";

print"<td>$kodstud</td><td>$fam</td><td>$imya</td><td>$otch</td><td>$gr_nomber
</td>";
    print"</tr>";
  $x++;
```

```
endwhile;
print "</table>";
mysql_close();
?>
</body>
</html>
```

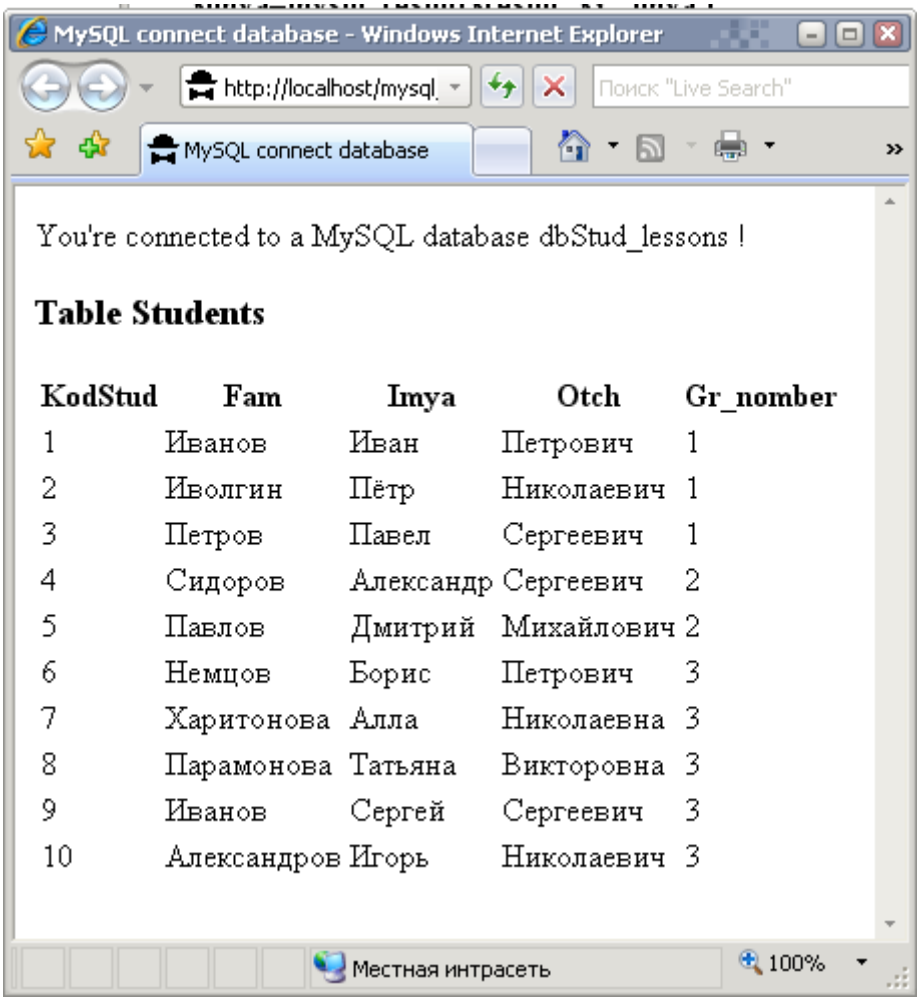

Рис. 6.6

mysql_affected_rows

Возвращает количество измененных записей в БД MySQL. Синтаксис**:**

int mysql_affected_rows([int link_identifier]);

Возвращает количество записей, которые были изменены в базе данных в результате выполнения запросов DELETE, INSERT, REPLACE или UPDATE. Если последним запросом была команда DELETE без ограничения WHERE (т.е. из таблицы были удалены все записи), то наша функция возвратит 0.

mysql_insert_id

Получает вставленный идентификатор. Синтаксис**:**

int mysql_insert_id([int $link_identifier])

Функция возвращает непосредственно перед ее вызовом сгенерированный идентификатор записи для автоинкрементного поля после выполнения команды insert. Вызывать ее разумно только сразу после выполнения инструкции insert, например, в таком контексте:

mysql_query("insert into Таблица(поле 1, поле 2) values("aa","bb")");
$id=mysql_insert_id();
Теперь к только что вставленной записи можно обратиться, используя идентификатор $id:
$r=mysql_query("select * from Таблица where id=$id");
$Row=mysql_fetch_array($r);

mysql_data_seek

Устанавливает указатель текущей строки. Синтаксис**:**

int mysql_data_seek(int result, int row_number)

Эта функция устанавливает указатель текущей строки в результате *result* в позицию *row_number*, так что следующий вызов **mysql_fetch_row()** и **mysql_fetch_array()** вернет значения полей именно этой строки. Нумерация записей ведется с 0. Возвращает false в случае ошибки или если строки кончились.